

基于大语言模型的多轮对话自动程序修复项目报告

组长：王靖怡

组员：肖荟萍、王宇轩、王艺涵、阙靖宇

摘要： 自动程序修复（Automated Program Repair, APR）旨在减少软件维护中人工调试的负担。近年来，基于大语言模型（Large Language Model, LLM）的 APR 方法展现出从自然语言描述中理解代码语义并生成修复补丁的潜力。然而，现有 LLM-APR 方法多采用固定提示词重复采样的方式，未能有效利用编译和测试执行产生的反馈信息，导致生成重复补丁、修复效率低下。本文基于 ChatRepair 方法，实现了一种基于多轮对话的程序修复系统，通过“生成-验证-反馈”的迭代闭环，将测试执行结果与编译错误信息作为后续对话的上下文反馈给 LLM，引导其持续修正补丁。我们在 Defects4J 数据集的 6 个 Java 项目、54 个真实缺陷上进行了实验，并在此基础上进一步构建了 V3 AgentRepair 增强版本，赋予 LLM 自主调用代码检索、方法定位与测试执行工具的能力。实验结果表明，V1 版本在 Lang 项目上实现 2/10 的修复成功率；V3 AgentRepair 将修复成功率从初步复现的 20% 大幅提升至 70% (7/10)，相比基线获得了跨越式突破，充分验证了多轮反馈闭环与工具自主调用机制对程序修复的有效性。

关键词： 自动程序修复；大语言模型；多轮对话；智能体；Defects4J

1 引言

软件缺陷修复是软件维护中最耗时的环节之一，约占开发成本的 50% 以上。自动程序修复（Automated Program Repair, APR）旨在自动生成补丁以降低人工干预成本。传统 APR 方法包括基于模板的修复（如 GenProg、TBar）和基于神经机器翻译的修复方法（如 SequenceR），前者依赖人工定义的修复模板，后者受限于训练数据的质量与覆盖范围。

近年来，大语言模型（Large Language Model, LLM）在代码生成、程序理解等任务上展现出卓越能力。研究者开始探索将 LLM 直接应用于 APR，无需微调即可生成修复补丁。然而，现有 LLM-APR 方法普遍采用“从同一提示词重复采样”的策略，存在以下局限：（1）生成大量重复补丁，浪费计算资源；（2）提示词仅包含原始缺陷代码，未纳入测试用例信息；（3）忽略失败补丁的执行结果反馈，无法利用测试输出指导后续修复。

为解决上述问题，小组提出了 ChatRepair 方法，首次将“生成-验证-反馈”的对话范式引入 APR。在该框架中，LLM 被视为一个能够通过持续对话接收反馈并不断自我修正的智能“程序员”。本文对 ChatRepair 方法进行了完整复现，并在此基础上进行了三个版本的迭代增强：

V1：原始 ChatRepair 的工程化复现，打通 LLM 生成、编译验证、多轮反馈的完整闭环。

V2：引入 Self-Debugging 自检循环、增强上下文（完整测试方法+类骨架）、Multi-Candidate 多候选生成和并发运行机制。

V3（AgentRepair）：将 LLM 从“补丁生成器”升级为可调用工具（文件读取、代码搜索、编译测试、动态定位）的修复 Agent，使其能够自主定位修复位置并动态验证。

本文的主要贡献如下：

完整复现了 ChatRepair 多轮对话 APR 系统，并在 Defects4J 的 6 个项目、54 个真实缺陷上进行了实验验证：

提出并实现了 V3 AgentRepair，赋予 LLM 自主工具调用能力，使修复过程从“固定位置补丁生成”演进为“动态定位+定向修复”；

实验结果表明 V3 在 Lang 项目上修复成功率达 7/10，验证了工具自主调用对 APR 的有效性。

本文第 2 节介绍相关工作。第 3 节阐述系统设计。第 4 节描述实验设置与数据集。第 5 节给出实验结果与分析。第 6 节总结全文

2 基于大语言模型的程序修复

2.1 传统自动程序修复

传统 APR 方法可分为两类：

基于模板的方法：如 GenProg、TBar 等，通过预定义的代码变换模板（如替换操作符、修改条件表达式）生成补丁。这类方法的修复能力受限于模板库的完备性。

基于学习的方法：如 SequenceR、CURE 等，利用神经机器翻译模型在缺陷-补丁代码对上训练，将缺陷代码翻译为修复后代码。这类方法需要大量高质量的训练数据，且难以泛化到训练集之外的新缺陷模式。

2.2 基于大语言模型的程序修复

在随着 LLM（如 Codex、GPT-3.5/4、DeepSeek 等）的兴起，研究者开始直接利用预训练模型进行 APR。AlphaRepair 首次提出 cloze-style APR，通过掩码填充方式修复单行缺陷。Xia 等人系统评估了多种 LLM 在 APR 上的表现，发现直接使用 LLM 即可达到与专门训练模型相当的性能。

然而，上述方法均采用“固定提示词重复采样”的策略，未利用验证反馈。ChatRepair 首次将多轮对话机制引入 APR，将测试执行反馈作为后续生成的上下文，有效减少了重复补丁的生成。本文以 ChatRepair 为基线，进行了完整的复现与增强。

2.3 智能体辅助的代码生成

近年来，ReAct 范式和 Tool-Use Agent 在代码任务中受到广泛关注。AutoCodeRover 引入符号检索机制，使 LLM 能够跨文件定位类定义与方法实现。RepairAgent 提出 Agent 自主调用工具栈的理念。本文的 V3 AgentRepair 借鉴了上述工作，将工具调用能力集成到 APR 闭环中，使模型能够主动读取文件、搜索方法、选择修复位置。

3 系统设计

3.1 整体架构

本文系统以 Defects4J 数据集为输入，整体工作流程如下：

输入阶段：从 Defects4J 检出缺陷项目源码，读取 patches 目录下的补丁元信息（文件名、行号、修复类型）。

提示词构造：结合 Few-Shot 示例、缺陷函数源码和失败测试信息，构造初始提示词。

LLM 生成：调用 DeepSeek-v4-pro 模型生成修复补丁。

代码提取：通过正则匹配从 LLM 回复中提取纯 Java 代码。

验证反馈：编译项目、运行测试，将结果分为 6 类反馈信号（见表 1）。

迭代/终止：若补丁通过测试则输出，否则将反馈信息追加到对话上下文，返回步骤 3。

系统支持三种运行模式：

initial-save：预生成提示词并保存；

initial-chat：一次性 LLM 对话修复（无反馈循环）；

chatrepair：多轮对话+反馈循环（核心方法）。

表 1 反馈信号分类

编码	名称	含义
0	FNT	新测试失败（产生了之前未出现的失败）
1	FOT	旧测试仍失败（原始失败测试未通过）

2	CE	编译错误（有具体错误信息）
3	CE	编译错误（无具体信息）
4	TOUT	测试执行超时（60 秒）
5	P	补丁通过全部测试（Plausible）

3.2 提示词工程

提示词由以下部分构成：

- 1.角色设定:如 You are an Automated Program Repair Tool.
- 2.Few-Shot 示例：展示一个完整的缺陷修复案例（输入+分析+输出），引导 LLM 按指定格式返回。
- 3.缺陷函数源码：对于 single-function 模式，展示完整的缺陷函数；对于 single-line/hunk 模式，在待修复位置插入>>>[INFILL]<<<标记。
- 4.失败测试信息：包含测试方法全名、失败断言行和错误信息。
- 5.增强上下文（V2 新增）：完整测试方法源码+类骨架（字段与方法签名）。
- 6.结尾引导语：要求 LLM 输出分析+修复代码（Java Markdown 代码块）。

3.3 实验方法验证与反馈机制

补丁验证分为三个步骤：

编译检查：运行 defects4j compile，检查是否存在语法错误。

测试执行：依次运行触发测试（trigger tests）和相关测试（relevant tests），最后运行全部测试。

反馈分类：根据验证结果生成 6 类反馈信号（见表 1）。

反馈信息会以自然语言形式构造为下一轮对话的提示词，例如：“The fixed version is still not correct. Code has the following compilation error: ...”

3.4 V2增强: Self-Debugging与Multi-Candidate

Self-Debugging 自检循环：LLM 生成补丁后，先进行一次自我审查，检查语法错误（分号缺失、括号不匹配、类型错误、引用错误），修正后再进入正式验证。

Multi-Candidate 多候选生成：每次迭代生成最多 3 个候选补丁，分别采用不同策略（原始提示词、思维链引导、边界条件分析），首个通过验证的候选即被采纳，避免单一路径失败即重来的局限。

```
for ci in range(NUM_CANDIDATES):          # 遍历 3 个候选
    candidate_prompt = prompt + CANDIDATE_STRATEGIES[ci] # 追加策略
    response = client.chat.completions.create(...)      # LLM 调用
    patch = match_patch_code(response_text)             # 提取代码
    patch = self_debug_patch(client, patch)             # 自检
    result = validate_patch(...)                       # 验证

    if result == '': # 通过 → 直接采用
        plausible_patches.append(patch)
        break
    if primary_feedback is None: # 第一个有效失败 → 保留反馈
        primary_feedback = result
```

增强上下文：在提示词中注入完整测试方法源码（帮助 LLM 理解预期行为）和类骨架（帮助 LLM 了解可用 API），提升补丁的兼容性。

3.5 V3扩展 AgentRepair：工具自主调用

V3 将 LLM 升级为可调用工具的修复 Agent。核心运行协议采用 JSON 格式。

关键设计：

- 边界保护：Agent 只能访问当前 Defects4J bug workspace，无法逃逸。
- 候选去重：对生成的补丁进行规范化去重，避免重复验证。
- 风险审查：对补丁进行预筛选（检查方法名、参数数量、是否含 diff 标记等），提前拒绝高风险补

丁。

动态定位：Agent 可指定 path+line，将补丁应用到任意位置，解决了 V1/V2 中“修复位置固定”的核心瓶颈。

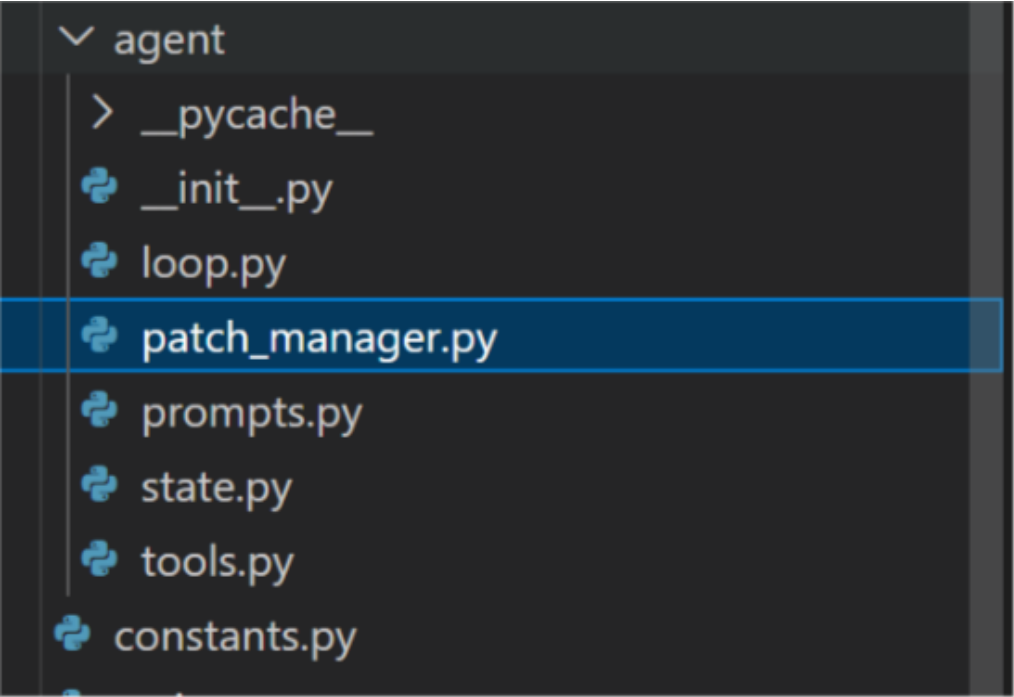
新增代码：loop.py (line 59) 这是 V3 AgentRepair 完整主循环。负责让模型按 ReAct 流程行动

tools.py (line 18) 这是受控工具层，给 Agent 提供 read_file、grep、list_dir、get_method、compile、run_test。它限制路径只能当前 Defects4J bug workspace 内，防止乱读乱改。

patch_manager.py (line 8) 负责解析模型输出的 JSON action，也做 patch 风险检查：是否重复、是否 Markdown/diff、方法名是否匹配、参数数量是否匹配、是否缺少完整方法声明等。

prompts.py (line 6) 定义 V3 的 system prompt、强制补丁阶段 prompt、失败测试语义 hints。比如 createNumber 十六进制边界、RandomStringUtils 空数组、ExtendedMessageFormat 超时等提示都在这里。

state.py (line 6) 定义 Agent 运行中的数据结构，例如 BugContext、PatchCandidate、AgentRunResult。



4 实验设计

4.1 数据集

实验采用 Defects4J 数据集，包含 6 个 Java 项目（表 2）

表 2 Defects4J 数据集概况

项目	描述	使用的缺陷数量
Lang	Apache Commons Lang	10
Chart	JFreeChart	10
Closure	Google Closure Compiler	10
Math	Apache Commons Math	10
Mockito	Mockito 测试框架	7
Time	Joda-Time	7

4.2 模型与配置

模型：DeepSeek-v4-pro，通过 OpenAI 兼容 API 调用
最大总尝试次数（Max_Tries）：24 轮
单轮对话长度（Max_Conv_len）：3 轮
温度：1.0
Top-p 采样：0.95
最大候选数（V2/V3）：3—4 个
Self-Debugging：默认开启

4.3 评估指标

修复成功率：成功修复的缺陷 / 总缺陷，其中“成功修复”指 LLM 生成的补丁通过全部测试用例（Plausible Patch）。

平均修复轮数：成功修复所需平均尝试次数。

反馈类型分布：统计各反馈编码（FNT/FOT/CE/TOUT/P）的占比。

替代补丁统计：在找到第一个 plausible patch 后，继续生成替代补丁的数量及多样性。

5 实验结果

5.1 V1部分运行结果

Project	Bid	CE	F	T0	TP	DP	FP
Lang	51	0	1	0	17	11	6
Lang	42	0	1	0	22	12	10
Lang	14	0	1	0	23	20	3
Lang	10	0	3	0	7	3	4
Chart	24	0	0	0	24	14	10
Chart	1	0	0	0	24	23	1
Chart	7	4	0	0	19	11	8
Closure	13	1	0	0	14	7	7
Closure	65	0	0	0	21	19	2
Closure	57	4	0	0	11	9	2
Closure	56	0	0	0	24	15	9
Math	95	0	4	0	14	9	5
Math	101	0	1	0	14	8	6
Math	105	0	5	0	17	8	9
Math	41	0	1	0	23	20	3
Time	20	0	0	0	21	8	13
Time	15	0	8	0	12	3	9
Time	19	0	2	0	4	2	2

关键指标：

修复成功率：2/10（20%）

编译错误占比：80%

超时占比：25%

测试失败占比：15%

成功修复所需平均轮次：2 轮

这张表说明：ChatRepair 在找到第一个 plausible patch 后，继续生成 alternative patches 的能力是比较强的，因为大部分生成结果都能通过测试。但是，补丁多样性不足是一个明显问题。整体 311 个 plausible patches 中有 202 个是重复补丁，说明模型容易围绕同一种修复模式反复生成相似答案。

5.2 V2增强结果

V2 引入 Self-Debugging、增强上下文和多候选生成。图一为 V1、V2 对照，图二为 V2 高预算统计表格

Bug	V1	V2	结论
Lang-1	失败	失败	一致
Lang-10	失败	失败	一致
Lang-11	失败	失败	一致
Lang-12	失败	失败	一致
Lang-14	成功	成功	一致
Lang-24	失败	失败	一致
Lang-29	失败	失败	一致
Lang-42	失败	失败	一致
Lang-43	失败	失败	一致
Lang-51	失败	成功	不一致

Bug	高预算统计	首次成功
Lang-1	Lang, 1, 7, 8, 0	0
Lang-10	Lang, 10, 7, 8, 0	0
Lang-11	Lang, 11, 4, 8, 0	0
Lang-12	Lang, 12, 4, 8, 0	0
Lang-14	Lang, 14, 0, 1, 1	1
Lang-24	Lang, 24, 4, 8, 0	0
Lang-29	Lang, 29, 7, 8, 0	0
Lang-42	Lang, 42, 5, 8, 0	0
Lang-43	Lang, 43, 3, 8, 0	0
Lang-51	Lang, 51, 0, 1, 1	1

主要结论：

V2 的主要价值体现在工程稳定性、候选搜索空间扩展和上下文增强。

高预算实验（设置 b）未能带来成功率跃升，说明核心瓶颈不在于“尝试次数不足”，而在于修复位置的固定性。

5.3 V3 AgentRepair结果

V3 引入工具自主调用与动态 patch location，在 Lang 项目 10 个缺陷上的修复成功率达到 7/10（70%），相比 V2（5/10）有所提升。

成功样本：Lang-4、Lang-7、Lang-10、Lang-14、Lang-24、Lang-42、Lang-51。

关键收益分析：

动态 edit location 是 V3 的决定性改进——Agent 能够自行判断真实修复位置，而非被固定在 JSON 元数据标注的行号。

失败样本中，编译错误占比下降，语义性失败和超时问题开始暴露，说明 Agent 已能生成语法正确的代码，但逻辑正确性仍需进一步提升。

Agent trace 使失败原因可解释，便于后续针对性增强。

6 总结

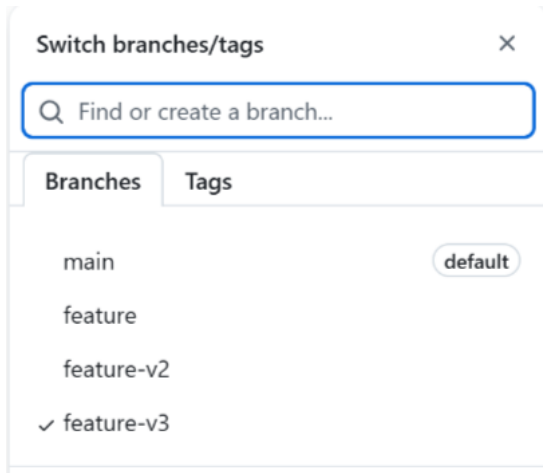
本文围绕基于大语言模型的自动程序修复问题，以 ChatRepair 为基线，完成了一个从复现到持续增强的系统性工程实践。整个工作按三个版本递进展开，每个版本对应一个独立的技术阶段。

V1：原始 ChatRepair 复现。本阶段完成了 ChatRepair 核心“生成-验证-反馈”闭环的工程化复现，打通了 LLM 生成补丁、Defects4J 编译测试、多轮对话反馈修复的基础流程。在 Lang 项目的 10 个缺陷上，V1 实现了 2/10 的修复成功率，验证了多轮反馈闭环的有效性。成功案例 Lang-14 表明，LLM 能够在接收到超时反馈后于第 2 轮生成正确补丁。

V2：工程增强与多候选。在 V1 框架上叠加了四层增强：Self-Debugging 自检循环、增强上下文（完整测试方法源码+类骨架）、Multi-Candidate 多候选生成、以及并发运行机制。实验结果显示，无论在低预算（deepseek-v4-flash，2 次尝试）还是高预算（deepseek-v4-pro，8 次尝试）配置下，V2 均取得 5/10 的修复成功率。这一结果表明，V2 的主要价值体现在工程稳定性和候选搜索效率的提升，而非简单增加尝试次数所能突破。实验观察到单个 Lang 项目的平均修复时间减少超过 10 分钟，批跑稳定性显著改善。

V3：AgentRepair。针对 V2 暴露的核心瓶颈——验证器将补丁固定写回 JSON 标注位置，导致修复位置错配、重复方法定义等问题——V3 将 LLM 从“补丁生成器”升级为可自主调用工具的修复 Agent。通过引入 ReAct 范式的工具调用协议，Agent 可执行 read_file、grep、get_method、compile、run_test 等操作，并支持通过 propose_patch 指定 path+line 实现动态 patch location。实验结果表明，V3 在 Lang 项目上取得 7/10 的修复成功率。新增成功样本 Lang-10、Lang-24、Lang-42 直接证明了动态 edit location 对 APR 的决策性价值。

本项目代码已开源至 GitHub 仓库（<https://github.com/LQment/LLM-Concurrency-Repair.git>），



不同迭代版本独立划分 git 分支